Adding React to Flask Part 2 (CodeGrade)

(https://github.com/learn-co-curriculum/python-p4-adding-react-to-flask-pt-2) (https://github.com/learn-co-curriculum/python-p4-adding-react-to-flask-pt-2/issues/new)

Learning Goals

- Use React and Flask together to build beautiful and powerful web applications.
- Organize client and server code so that it is easy to understand and maintain.

Key Vocab

- **Full-Stack Development**: development of a frontend and a backend for an application. True full-stack development includes a database, a logic/server layer, and a frontend built in JavaScript, HTML, and CSS.
- **Backend**: the layer of a full-stack application that handles business logic and other programmatic tasks that users do not or should not see. Can be written in many languages, including Python, Java, Ruby, PHP, and more.
- **Frontend**: the layer of a full-stack application that users see and interact with. It is always written in the frontend languages: JavaScript, HTML, and CSS. (*There are others now, but they are based on these three.*)
- **Cross-Origin Resource Sharing (CORS)**: a method for a server to indicate any ports (or other identifiers) for servers that can share its resources.
- **Transmission Control Protocol (TCP)**: a protocol that defines how computers send data to each other. A connection is formed and stays active until the applications on either end have finished sending data to one another.
- **Hypertext Transfer Protocol (HTTP)**: a stateless protocol where applications communicate for the length of time that it takes for data to be transferred.
- **Websocket**: a protocol that allows clients and servers to communicate with one another in both directions. The bidirectional nature of websocket communication allows a connected state to be generated and the connection maintained until it is terminated by one side. This

allows for speedy and seamless connections between frontends and backends.

Introduction

Now that we've built several Flask backends and reviewed what goes into a React frontend, let's build a new full-stack application from scratch.

Generating a React Application

To get started, let's spin up our React application using `create-react-app`:

```
$ npx create-react-app client --use-npm
```

This will create a new React application in a `client` folder, and will use npm instead of yarn to manage our dependencies.

When we're running React and Flask in development, we'll need two separate servers running on different ports — we'll run React on port 4000, and Flask on port 5555. Whenever we want to make a request to our Flask API from React, we'll need to make sure that our requests are going to port 5555.

We can simplify this process of making requests to the correct port by using `create-react-app` in development to **proxy the requests to our API (https://create-react-app.dev/docs/proxying-api-requests-in-development/)**. This will let us write our network requests like this:

```
fetch("/movies");
// instead of fetch("http://127.0.0.1:5555/movies")
```

To set up this proxy feature, open the `package.json` file in the `client` directory and add this line at the top level of the JSON object:

```
"proxy": "http://127.0.0.1:5555"
```

Let's also update the "start" script in the the `package.json` file to specify a different port to run our React app in development:

```
"scripts": {
  "start": "PORT=4000 react-scripts start"
}
```

Creating the Server Application

With that set up, let's get started on our Flask app. In the root directory, run `pipenv install && pipenv shell` to create and enter your virtual environment.

Create an `app.py` in the `server/` directory and enter the following code:

```
# server/app.py

from flask import Flask, make_response, jsonify
from flask_cors import CORS

app = Flask(__name__)

CORS(app)

@app.route('/movies', methods=['GET'])
def movies():
    response_dict = {
        "text": "Movies will go here"
    }

    return make_response(jsonify(response_dict), 200)

if __name__ == '__main__':
    app.run(port=5555)
```

From the server directory, run:

```
$ python app.py
```

Then, open a new terminal, and run React in the `client/` directory:

```
$ npm start
```

Verify that your app is working by visiting:

- **http://127.0.0.1:4000 (http://127.0.0.1:4000)** to view the React application.
- **http://127.0.0.1:5555/movies (http://127.0.0.1:5555/movies)** to view the Flask application.

We can also see how to make a request using `fetch()`. First though, let's populate our database and update the Flask app to show that data. Modify `server/app.py` as follows:

```
# server/app.py
#!/usr/bin/env python3


from flask import Flask, request, make_response, jsonify
from flask_cors import CORS
from flask_migrate import Migrate

from models import db, Movie

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.json.compact = False

CORS(app)
migrate = Migrate(app, db)
```

```
db.init_app(app)

@app.route('/movies', methods=['GET'])
def movies():
    if request.method == 'GET':
        movies = Movie.query.all()

        return make_response(
            jsonify([movie.to_dict() for movie in movies]),
            200,
        )

    return make_response(
        jsonify({"text": "Method Not Allowed"}),
        405,
    )

if __name__ == '__main__':
    app.run(port=5555)
```

Once you have saved these changes, run `flask db upgrade` to generate your database and `python seed.py` to fill it with data.

```
$ flask db upgrade
$ python seed.py
```

In the React application, update your `App.js` file with the following code:

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
```

```
      fetch("/movies")
        .then((r) => r.json())
        .then((movies) => console.log(movies));
    }, []);

    return <h1>Check the console for a list of movies!</h1>;
  }

  export default App;
```

This will use the `useEffect` hook to fetch data from our Flask API, which you can then view in the console.

Running React and Flask Together

Since we'll often want to run our React and Flask applications together, it can be helpful to be able to run them from just one command in the terminal instead of opening multiple terminals.

To facilitate this, we'll use the excellent **Honcho (https://honcho.readthedocs.io/en/latest/)** module. We included it in your Pipfile; if you wanted to add it yourself, you would run:

```
$ pipenv install honcho
```

Honcho works with a special kind of file known as a Procfile, which lists different processes to run for our application. Some hosting services, such as Heroku, use a Procfile to run applications, so by using a Procfile in development as well, we'll simplify the deploying process later.

In the root directory, create a file `Procfile.dev` and add this code:

```
web: PORT=4000 npm start --prefix client
api: gunicorn -b 127.0.0.1:5555 --chdir ./server app:app
```

Rather than running on a development server, we're migrating to **Gunicorn (https://flask.palletsprojects.com/en/2.2.x/deploying/gunicorn/)**, a simple Python WSGI server which will be much more capable of handling requests when we deploy our application to the internet.

> **NOTE: Gunicorn will run on WSL, but not Windows. Waitress (https://docs.pylonsproject.org/projects/waitress/en/latest/) is a good alternative if you cannot use MacOS, Linux, or WSL.**

Now, run your Procfile with Honcho:

```
$ honcho start -f Procfile.dev
```

This will start both React and Flask on separate ports, just like before; but now we can run both with one command!

There is one big caveat to this approach: by running our client and server in the same terminal, it can be more challenging to read through the server logs and debug our code. If you're doing a lot of debugging in the terminal, you should run the client and server separately to get a cleaner terminal output.

Conclusion

In the past couple lessons, we've seen how to put together the two pieces we'll need for full-stack applications by creating a Flask API in a `server/` directory, and `create-react-app` to create a React project in a `client/` directory. Throughout the rest of this module, we will focus on how clients and servers communicate and how we can improve upon their connection.

Check For Understanding

Before you move on, make sure you can answer the following questions:

▶ *1. What options do you have for running Flask and React at the same time?*

▶ *2. What are the advantages and disadvantages of using Honcho as described in this lesson?*

Resources

- Proxying API Requests in Create React App (https://create-react-app.dev/docs/proxying-api-requests-in-development/)
- Honcho: manage Procfile-based applications - honcho (https://honcho.readthedocs.io/en/latest/)
- Gunicorn - Pallets (https://flask.palletsprojects.com/en/2.2.x/deploying/gunicorn/)

This tool needs to be loaded in a new browser window

Load Adding React to Flask Part 2 (CodeGrade) in a new window